

The people schema — applicants, members, and investors

Purpose. This is the lookup reference for `people`, the Postgres schema that records everyone connected to AlgoGators: applicants who filled in a recruiting form, members on a team, and (eventually) investors. Use it to find out what a table holds, why a column is shaped the way it is, and what the database will and will not let you write.

Applies to. Anyone reading from or writing to the platform database — the applicant tracking system (ATS), the Excel round-trip tooling, the Microsoft Forms ingest, the website roster, or a one-off query. It describes the schema **as committed in** `001_people_schema.sql`, which has been proven to execute against a throwaway Postgres in CI but **has not been applied to any real database**. It does not describe the ATS application itself, the ingest scripts, or the market-data schemas (`futures_data`, `equities_data`, `options_data`, `synthetic`) which live in `services/data-ngin` and have nothing to do with people.

Before you start

- You can read SQL. You do not need to have written a Postgres migration before — the recurring patterns are glossed in [Patterns used throughout](#).
- Docker, to run the schema locally. Everything below can be checked with `just test libs/platform-db` from the repo root.
- The schema lives at `libs/platform-db/migrations/001_people_schema.sql` on branch `feat/people-schema`. It is in `libs/platform-db` rather than in a service because people are shared plumbing, not one service's data.

What this schema is for

Three different things write to these tables, and the schema is shaped by that fact more than by anything else:

1. **A Microsoft Forms export**, ingested as `.xlsx` once per recruiting cycle.
2. **An Excel round-trip** — someone exports a sheet, edits it by hand, uploads it back.
3. **People clicking around** in an application, and an ATS with its own database login.

None of them can be trusted to be the only writer, and none of them can be trusted to have read the newest version of a row. Two structural decisions follow, and they explain most of what looks unusual in the table definitions.

Patterns used throughout

These five patterns appear on nearly every table. Read this section once and the rest of the doc gets much shorter.

Pattern	What it means	Why it is there
<code>row_version INT</code>	Every mutable table has one. An update must name the version it read: <code>UPDATE ... WHERE id = \$1 AND row_version = \$3</code> . Zero rows	Two people export the same sheet an hour apart, both edit, both upload. Without this the second

Pattern	What it means	Why it is there
	updated means someone else changed the row first, and the write is rejected rather than applied.	upload silently erases the first — no error, no conflict, no trace. This is called <i>optimistic concurrency</i> : writes are allowed to race, and the loser is told.
deleted_at TIMESTAMPZ	Nothing is hard-deleted by any automated path. "Deleting" sets this timestamp; restoring sets it back to NULL .	A real DELETE from an Excel upload is unrecoverable. A hard delete stays a deliberate manual act by a database owner.
Partial unique indexes	A uniqueness rule written as CREATE UNIQUE INDEX ... WHERE deleted_at IS NULL — it applies only to live rows.	A plain UNIQUE would let a soft-deleted person keep owning their email address forever, so re-adding them would fail against a row the application cannot even see.
Composite foreign keys	Some tables reference <i>two</i> columns of a parent at once, e.g. member references student (person_id, id) .	Two separate FKs would each be individually valid while the row as a whole is a lie — a member row naming person 7 and a student record belonging to person 19.
Generated columns	Columns computed by the database from other columns and stored (name_key , grad_sort , stage_rank , application_score.total). They cannot be written to directly.	A derived value maintained by application code eventually disagrees with what it derives from. This makes that impossible.

Two conventions worth naming:

- **Enums are TEXT + CHECK (x IN (...))**, never **CREATE TYPE ... AS ENUM**. Altering a real Postgres enum takes a migration-time lock, and removing a value needs a full type rewrite.
- **All time is TIMESTAMPTZ**. No naive timestamps anywhere.

The tables

Fourteen tables, in dependency order — the order the migration must create them in, because several foreign keys point forward.

people.team — the lookup of teams

A lookup table rather than a **CHECK** constraint on a column, because team names change and a **CHECK** would make every rename a migration.

Column	Type	Meaning
id	BIGSERIAL PK	
name	TEXT NOT NULL	Display text. Free to change.

Column	Type	Meaning
slug	TEXT NOT NULL UNIQUE	The stable handle, matching <code>^[a-z][a-z0-9_]*\$</code> . Code, the Excel sheet, and the form value mapping all key on this.
active	BOOLEAN NOT NULL	Default <code>TRUE</code> .
deleted_at, row_version, created_at, updated_at		Standard set.
- slug is inline UNIQUE, not partial: a retired team keeps its slug reserved, because reusing one would silently re-point historical assignments at a different team. - team_name_lower_idx is a partial unique index on <code>lower(name)</code> — a retired team's <i>display name</i> can be reused, its slug cannot. - Seeded with four rows: <code>quant_research</code>, <code>quant_dev</code>, <code>quant_trading</code>, <code>investor_relations</code>.		

people.person — one row per human

The identity table. Everyone else in this schema hangs off it.

Column	Type	Meaning
id	BIGSERIAL PK	
first_name, last_name	TEXT NOT NULL	Both non-blank.
email	TEXT NOT NULL	The applicant's UF email , which is what the forms ask for.
person_type	TEXT, nullable	'member' or 'investor'.
name_key	generated, stored	<code>`lower(trim(first</code>
deleted_at, row_version, created_at, updated_at		Standard set.
person_type is nullable on purpose. An applicant is neither a member nor an investor, and <code>NOT NULL</code> would force mislabelling every applicant as one of them on insert. person_type is denormalized and is not the source of truth. The authoritative answer to "is this person a member?" is the existence of a <code>people.member</code> row. Anything gating access on membership must join <code>member</code> rather than read this column. <code>person_email_lower_idx</code> is unique on <code>lower(email)</code> and partial. Case-insensitive because a plain <code>UNIQUE</code> lets <code>Jane@uf1.edu</code> and <code>jane@uf1.edu</code> both exist, which is how one person becomes two rows and their application history splits in half. name_key is deliberately not unique. Two people can share a name, and forbidding it would reject a real applicant to simplify one internal tool. Email is the identity; the name is a label. It exists to pair manually uploaded resume files back to people, and same-name collisions are reported for a human to resolve — permanently and by design.		

people.student — the academic record

Column	Type	Meaning
id	BIGSERIAL PK	
person_id	BIGINT NOT NULL → person	ON DELETE CASCADE.
gpa	NUMERIC(3,2), nullable	Range 0–4.5.
class_standing	TEXT, nullable	"Sophomore", etc. Self-reported free text.
grad_term	TEXT NOT NULL	'spring', 'summer', or 'fall'.
grad_year	SMALLINT NOT NULL	1900–2200.
grad_sort	generated, stored	$\text{grad_year} * 10 + \text{term rank}$, so "graduating before X" is one comparison.
deleted_at, row_version, created_at, updated_at		Standard set.

- **GPA is nullable because neither form asks for it.** It is hand-entered or absent. **NUMERIC** rather than a float so 3.85 round-trips exactly; the ceiling is 4.5 rather than 4.0 because weighted scales exceed 4.
- **Graduation is a term and a year, not a date.** A **DATE** would force inventing a day of the month, and 'fall 2026' as free text cannot be range-queried. There is no winter term — the club does not have one.
- **UNIQUE (person_id, id)** exists so **member** can reference the pair. **student_person_live_idx** separately enforces one *live* student row per person.

people.student_major — majors and minors

Its own table rather than two **TEXT[]** columns, so a student can hold any number of each and "who studies finance" is an index hit rather than an array scan.

Column	Type	Meaning
id	BIGSERIAL PK	
student_id	BIGINT NOT NULL → student	ON DELETE CASCADE.
field	TEXT NOT NULL	Free text, stored as typed.
kind	TEXT NOT NULL	'major' or 'minor'.
deleted_at, row_version, created_at, updated_at		Standard set.

- Unique per (**student_id**, **lower(field)**, **kind**), live rows only — case-insensitive, because a case-sensitive key would let "Finance" and "finance" both insert for the same student.

- **Known cost:** "CS", "Comp Sci", and "Computer Science" are three different majors to any query. Normalizing that belongs in an import-time pass, not a **CHECK** constraint.

people.attachment — resumes and strategy write-ups

One table with a **kind** rather than two near-identical tables.

Column	Type	Meaning
id	BIGSERIAL PK	
person_id	BIGINT NOT NULL → person	Anchored to person, not student.
kind	TEXT NOT NULL	'resume' or 'trading_strategy'.
source_url	TEXT, nullable	The SharePoint link from the export.
original_filename	TEXT NOT NULL	Exactly as Forms produced it. Never normalized.
content	BYTEA, nullable	The bytes.
content_type	TEXT, nullable	'application/pdf' only.
size_bytes	BIGINT, nullable	> 0 and ≤ 10 MB.
sha256	TEXT, nullable	64 lowercase hex characters.
fetches_at	TIMESTAMPTZ, nullable	When the bytes arrived.
deleted_at, row_version, uploaded_at, updated_at		Standard set (uploaded_at in place of created_at).

- **Anchored to person, not student**, because an applicant submits a resume before there is any reason to have a student row — and **student** requires graduation info, so anchoring here would force inventing a graduation term to accept an upload.
- **content is nullable, and this is the normal state after ingest.** The row is created by the xlsx ingest before any file exists; the file is uploaded by hand later. **attachment_unfetched_idx** indexes exactly those rows — it is the "who to chase before a review round" report.
- **CHECK ((content IS NULL) = (fetches_at IS NULL))** keeps a row from claiming a fetch that produced nothing, or holding bytes with no record of when they arrived.
- **original_filename is never cleaned up.** Forms appends **_{respondent name}** before the extension, so the matcher parses from the right: strip the extension, take the substring after the last underscore, normalize, compare against **person.name_key**. Any normalization here would destroy the only link back to a person.
- The 10 MB ceiling matches Microsoft Forms' smallest per-file setting. Deliberately not stricter: a lower ceiling would reject a file the form accepted, on a submission nobody can redo.
- **UNIQUE (person_id, id)** exists so **application** can reference the pair.

people.member — who is in the club

Column	Type	Meaning
<code>id</code>	<code>BIGSERIAL</code> PK	
<code>person_id</code>	<code>BIGINT</code> NOT NULL → <code>person</code>	
<code>student_id</code>	<code>BIGINT</code> , nullable	Part of a composite FK to <code>student</code> (<code>person_id</code> , <code>id</code>).
<code>is_leadership</code>	<code>BOOLEAN</code> NOT NULL	Default <code>FALSE</code> .
<code>joined_on</code>	<code>DATE</code> NOT NULL	Default today.
<code>left_on</code>	<code>DATE</code> , nullable	<code>NULL</code> means currently active.
<code>deleted_at</code> , <code>row_version</code> , <code>created_at</code> , <code>updated_at</code>		Standard set.

- **There is no `team` column here.** People move teams, and a column would overwrite the old assignment and lose the history. See `member_team`.
- **`is_leadership` is orthogonal to team, not a team of its own.** A lead belongs to a real team as well, which is why the same person appears under two headings on the roster page.
- **`student_id` is nullable** for members who are not students (alum, staff, advisor). Postgres treats a composite FK with any `NULL` column as satisfied, so this needs no separate exemption.
- **Alumni stay in the table** with `left_on` set, rather than being deleted out of it.
- `member_person_live_idx`: one live member row per person.

`people.member_team` — team assignment history

Column	Type	Meaning
<code>id</code>	<code>BIGSERIAL</code> PK	
<code>member_id</code>	<code>BIGINT</code> NOT NULL → <code>member</code>	<code>ON DELETE CASCADE</code> .
<code>team_id</code>	<code>BIGINT</code> NOT NULL → <code>team</code>	
<code>started_on</code>	<code>DATE</code> NOT NULL	
<code>ended_on</code>	<code>DATE</code> , nullable	<code>NULL</code> means this is the current assignment.
<code>deleted_at</code> , <code>row_version</code> , <code>created_at</code> , <code>updated_at</code>		Standard set.

- `member_team_current_idx` is unique on `member_id` **where `ended_on` IS NULL** — one *current* team per member, with past assignments unconstrained. A plain `UNIQUE (member_id)` would forbid the history, which is the thing this table exists for.
- A member with **no** open row is a legitimate temporary state meaning "accepted, not yet placed".

people.application_form — the form of record, per cycle

One row per cycle per track. The form URL changes every cycle, and the database must not learn form identity from a website redeploy.

Column	Type	Meaning
id	BIGSERIAL PK	
cycle_term	TEXT NOT NULL	'fall' or 'spring' only.
cycle_year	SMALLINT NOT NULL	
track	TEXT NOT NULL	'analyst' or 'relations'.
form_url	TEXT NOT NULL	
external_form_id	TEXT, nullable	
question_map	JSONB NOT NULL	Which question feeds which column, for <i>this</i> cycle's form.
opened_at, closed_at	TIMESTAMPTZ, nullable	closed_at >= opened_at when both are set.
deleted_at, row_version, created_at, updated_at		Standard set.

- **cycle_term is narrower than student.grad_term** — recruiting runs fall and spring only, while people do graduate in summer.
- **track is not a team.** One 'analyst' form covers Quant Research, Quant Development, and Quant Trading, and asks applicants to rank the three. Relations is its own form.
- **question_map is data, not code**, because question wording changes between cycles and a rename must not require a deploy. An entry carries a column name and optionally a transform and a value lookup:

```
{
  "UF Email": {
    "column": "person.email",
  },
  "Graduation Semester": {
    "column": "student.grad_term",
    "transform": "lower",
  },
  "Which team are you most interested in joining?": {
    "column": "application_team_preference",
    "transform": "ranked_list",
    "values": {
      "Quant Dev": "quant_dev",
      "Quant Trading": "quant_trading",
      "Quant Research": "quant_research"
    }
  }
}
```

The value map matters: the form says "Quant Dev" and **people.team** says "Quantitative Development", and a lookup that matches nothing looks exactly like an unanswered question.

- Older analyst forms asked for a month and year ("May, 2029") rather than a season. That cycle's row must keep its `month_year_to_term` transform, or re-ingesting its export silently produces `NULL` graduation terms.
- Unique per `(cycle_term, cycle_year, track)`, live rows only.

people.application — one submission

Column	Type	Meaning
<code>id</code>	<code>BIGSERIAL PK</code>	
<code>person_id</code>	<code>BIGINT NOT NULL → person</code>	
<code>cycle_term, cycle_year</code>		Same constraints as <code>application_form</code> .
<code>form_id</code>	<code>BIGINT → application_form</code>	
<code>external_response_id</code>	<code>TEXT</code> , nullable	The export's <code>Id</code> column.
<code>raw_response</code>	<code>JSONB</code> , nullable	The whole response, verbatim.
<code>stage</code>	<code>TEXT NOT NULL</code>	'submitted', 'screen', 'interview', 'final'.
<code>stage_rank</code>	generated, stored	1–4, for ordering.
<code>outcome</code>	<code>TEXT NOT NULL</code>	'pending', 'accepted', 'rejected', 'withdrawn'.
<code>resume_attachment_id</code>	<code>BIGINT</code> , nullable	Composite FK to <code>attachment (person_id, id)</code> .
<code>submitted_at</code>	<code>TIMESTAMPTZ NOT NULL</code>	
<code>deleted_at, row_version, created_at, updated_at</code>		Standard set.

- **stage and outcome are separate on purpose.** One status enum cannot say "was rejected, and got as far as the interview" — the value would have to be either `interview` or `rejected`, losing the other half. `stage` holds the *furthest stage reached*, only moves forward, and is not reset when the outcome lands.
- **external_response_id is not globally unique.** Forms numbers responses per form starting at 1, so the analyst form's response 1 and the relations form's response 1 are two different people, and next cycle's forms start at 1 again. Uniqueness is `(form_id, external_response_id) WHERE external_response_id IS NOT NULL`.
- **That index is deliberately *not* partial on deleted_at** — the opposite of the email rule. A response id names one event that happened once, so re-importing must find a soft-deleted row and restore it rather than create a second.
- `application_person_cycle_idx` allows one *live* application per person per cycle, so withdrawing and re-applying in the same cycle stays possible.

- **raw_response is never parsed at query time** — the mapped columns are what queries touch. It exists so a question added or reworded mid-cycle can be backfilled into a new column instead of asking applicants to resubmit. It also holds date of birth, which has no column on purpose.
- **resume_attachment_id pins the resume this application was judged on**, so a later upload does not retroactively change the record of a past cycle.

people.application_team_preference — the ranked answer

Column	Type	Meaning
application_id	BIGINT NOT NULL → application	ON DELETE CASCADE.
team_id	BIGINT NOT NULL → team	
rank	SMALLINT NOT NULL	≥ 1.
created_at	TIMESTAMPTZ NOT NULL	

Primary key (application_id, team_id), plus UNIQUE (application_id, rank) so no two teams share a rank within one application. No deleted_at or row_version — rows are replaced with their parent application, not edited independently.

- Exists because the analyst form asks applicants to **rank** the three analyst teams, and the export carries an ordered semicolon-delimited list ("Quant Dev;Quant Trading;Quant Research;") that a single FK cannot hold.
- **Relations applications get no rows here at all** — Investor Relations has no sub-divisions. Which team a relations applicant applied to is answered by application_form.track.
- **A preference is not a placement.** Rank 1 records what somebody asked for; where they end up is member_team, decided after acceptance. The two are allowed to disagree, and rank is never an entitlement.

people.application_score — the six-criterion rubric

One consolidated grade per application. Six SMALLINT NOT NULL columns, each BETWEEN 1 AND 10:

Criterion	Judges
economic_foundation	Strength of the economic hypothesis underpinning the strategy.
innovation	Creativity and distinctiveness versus established frameworks.
alpha_potential	Evidence of excess returns and a clear implementation path.
risk_management	Depth of the plan to mitigate identified risks.
liquidity_capital	Whether the strategy is practical in real markets.
performance_evidence	Rigour of the historical or alternative evidence offered.

Plus notes TEXT, scored_at, the standard deleted_at/row_version/created_at/updated_at, and total — a generated column summing the six, range 6–60. The full 1-3 / 4-6 / 7-9 / 10 band descriptions for each criterion are in the migration's own comments.

- **Columns rather than a JSONB blob or a row-per-criterion table.** The rubric is a fixed published list, which makes it schema, not data. Columns give each criterion its own **CHECK** (a blob cannot reject `{"innovation": 47}` or a typo'd key), make **NOT NULL** mean "this grade is complete", let **total** be generated, and make **AVG(alpha_potential)** across a cycle a plain query.
- Row-per-criterion was rejected because it cannot enforce "all six present": nothing stops a set of four rows, and **total** becomes a **SUM** that silently returns a smaller number instead of erroring.
- **The cost, accepted knowingly:** a seventh criterion is a migration, and existing rows need a decision about their value for it.
- **There is no grader column.** Applications are graded by leadership as a body, not by an individual, so a **scored_by** field would name one person for a decision several people made. **change_log.actor** still records who entered the row, which is the part worth auditing.
- One live score per application, via a partial unique index.

people.investor — a deliberate stub

person_id primary key referencing **person**, plus **deleted_at**, **row_version**, **created_at**, **updated_at**. Nothing else.

Not designed yet, on purpose: the shape depends on whether investors are individuals or entities, and whether commitments and positions belong here or in a finance schema. Filling it in is a later migration.

people.import_batch — one row per write event

An Excel upload is one batch; one form sync is one batch; a hand edit in the app is one batch. This is what makes "undo that upload" a real operation rather than an archaeology project.

Column	Type	Meaning
id	BIGSERIAL PK	
source	TEXT NOT NULL	'form_sync', 'excel_upload', 'app_edit', 'api'.
filename	TEXT , nullable	For file-based sources.
file_sha256	TEXT , nullable	64 hex chars. Catches "did I already upload this exact sheet" before any row is touched.
actor	TEXT NOT NULL	Who did it.
note	TEXT , nullable	
row_count	INTEGER NOT NULL	
status	TEXT NOT NULL	'pending', 'applied', 'failed', 'reverted'.
started_at , finished_at , reverted_at , reverted_by		

actor must come from the authenticated session, never from a field in the uploaded workbook. Anyone can upload, so this record is the only accountability there is — and a self-declared actor would make it worthless, since whoever uploads would choose what it says.

people.change_log — row-level history

Every insert, update, and soft delete on a logged `people.*` table lands here with the whole row before and after.

Column	Type	Meaning
<code>id</code>	BIGSERIAL PK	
<code>batch_id</code>	BIGINT, nullable → <code>import_batch</code>	
<code>table_name</code>	TEXT NOT NULL	
<code>row_id</code>	BIGINT NOT NULL	
<code>operation</code>	TEXT NOT NULL	'insert', 'update', 'soft_delete', 'restore'.
<code>before, after</code>	JSONB, nullable	Whole-row snapshots.
<code>actor</code>	TEXT NOT NULL	
<code>occurred_at</code>	TIMESTAMPTZ NOT NULL	

Whole-row snapshots rather than per-column diffs: storage is cheap at this scale, and a whole row is what you need to actually put something back. Reverting a batch is reading `change_log` for that `batch_id` newest-first and applying each entry's `before`.

Triggers

Two trigger functions, applied by a `DO` block so the migration stays re-runnable.

Trigger	Fires	On
<code><table>_log_change</code>	AFTER INSERT OR UPDATE	10 tables: <code>person</code> , <code>student</code> , <code>student_major</code> , <code>member</code> , <code>member_team</code> , <code>team</code> , <code>application</code> , <code>application_score</code> , <code>application_form</code> , <code>investor</code>
<code><table>_bump_version</code>	BEFORE UPDATE	Those 10 plus <code>attachment</code> — 11 in total

- **History is written by a trigger rather than by application code on purpose.** `updated_at` is app-maintained and a stale one is cosmetic; history is not. In application code, a `psql` session, a migration, or a second service would write rows with no record, and the gap would be invisible until someone needed it. A trigger catches every path by construction.
- **row_version gets the same treatment.** The `WHERE row_version = $n` guard in an `UPDATE` is what *detects* a conflicting write; the trigger is what guarantees the number actually moved. Left to application code, one caller that forgot the bump would silently disarm optimistic concurrency for every Excel user.

- **attachment is deliberately excluded from change logging** — `to_jsonb` would base64-encode the whole PDF into a history row on every write. It still gets the `row_version` trigger, since that column is what the Excel round-trip depends on.
- The trigger reads two session settings when present: `people.actor` (falling back to `current_user`) and `people.batch_id`. A writer that can only speak SQL should set both per transaction.
- `CREATE TRIGGER` has no `IF NOT EXISTS` and `CREATE OR REPLACE TRIGGER` is Postgres 14+, so the block does `DROP` then `CREATE` — idempotent on every version.

Views and access

One view: `people.roster_public`. Every live member — past and present — with `id`, `first_name`, `last_name`, current team name, `is_leadership`, `joined_on`, and an `active` flag derived from `left_on IS NULL`. Alumni appear with `active = false` rather than being filtered out. No email, no GPA, no resume, no date of birth.

Grants are wrapped in an existence check, because these roles do not exist in every target database yet and an unguarded `GRANT` to a missing role aborts the whole migration:

Role	Gets
<code>db_readwrite_all</code>	<code>USAGE</code> on the schema, <code>ALL</code> on all tables, <code>USAGE</code> on all sequences.
<code>db_readonly</code>	<code>USAGE</code> on the schema and <code>SELECT</code> on <code>roster_public</code> only — never the base tables.

Until those roles exist, the schema is reachable only by the database owner. That is the intended interim state, and tightening it later is a second migration rather than a rewrite. GPA, resumes, rubric grades, and the dates of birth inside `raw_response` are the most sensitive rows in the system, so this should not be left indefinitely.

Out of scope

Facts about what this schema deliberately does **not** do. Each is a design decision, not an oversight.

Not in scope	Instead
Automatic file fetching. Nothing downloads resumes from SharePoint — that would need Graph API credentials against a personal university account.	<code>attachment.content</code> stays <code>NULL</code> until a human uploads the file; <code>source_url</code> is the cross-check that the right file was paired.
A date-of-birth column , now or later.	It stays inside <code>raw_response</code> , optional, and no ingest step fails without it.
GPA collection. Neither recruiting form asks.	<code>student.gpa</code> is nullable and hand-entered.
Per-grader scores.	One consolidated <code>application_score</code> row; leadership grades as a body.
Investor modelling.	A stub table, pending a decision on individuals vs. entities.

Not in scope	Instead
Hard deletes by any automated path.	Soft delete via <code>deleted_at</code> ; a real erasure is a deliberate manual act, and has to reach <code>change_log</code> rows too.
Market data of any kind.	<code>services/data-ngin</code> owns <code>futures_data</code> , <code>equities_data</code> , <code>options_data</code> , <code>synthetic</code> .
Application code — the ATS, the ingest scripts, the Excel diff tooling.	This document covers the tables those things write to.
Any real database. The migration has never been applied outside a throwaway container.	Running it for real is a deliberate, separate act.

Verify it worked

From the repo root:

```
just test libs/platform-db
```

Brings up a throwaway Postgres in Docker, applies every migration in filename order, and runs the suite. Expect **68 passed** at 99% coverage. The tests assert the claims above that a reader would otherwise have to take on faith — that a partial index really does free an email on soft delete, that `row_version` really is bumped by the database rather than the caller, that a composite FK really does reject a member row pointing at another person's student record.

To confirm the migration is safe to re-run, from `libs/platform-db`:

```
just migrate-idempotent
```

Applies every file twice. Any error on the second pass is a real defect. Both recipes need Docker, and on Windows, Git Bash.

Related

- `libs/platform-db/README.md` — what else the library owns, and why it is not a dependency of `libs/algosystem`.
- `libs/platform-db/migrations/001_people_schema.sql` — the schema itself. Every design decision summarized here is argued at length in its comments.
- `libs/platform-db/tests/test_people_schema.py` — the executable version of this document.